Viva Preparation - Member 4 - Notification Service

Owned Component

Owned microservice:

- `notification-service`

Main files to show:

- `services/notification-service/src/app.js`
- `services/notification-service/src/controllers/notificationController.js`
- `services/notification-service/src/routes/notificationRoutes.js`
- `services/notification-service/src/models/Notification.js`
- `services/notification-service/Dockerfile`
- `sonar/notification-service.properties`

What To Say

I implemented the notification service. This service stores and returns in-app notifications for customers, organizers, and admins.

The notification service is triggered by booking activity. For example, when a customer creates a booking request, the booking service sends notification requests so the customer, organizer, and admin can see updates.

Endpoints To Explain

- `POST /api/notifications/send` - creates a notification.
- `GET /api/notifications/user/:userId` - lists notifications for a user.
- `GET /api/notifications/:id` - gets a notification by ID.
- `PATCH /api/notifications/:id/read` - marks a notification as read.

Through the gateway these become:

- `/notifications/user/:userId`
- `/notifications/:id/read`

Integration Point

Integration to demonstrate:

`booking-service` calls `notification-service` when booking actions happen.

Example explanation:

When a booking is created, confirmed, rejected, or cancelled, the booking service sends a request to notification-service. The notification service stores the alert so the correct user can see it in the frontend.

Demo Steps

1. Log in as customer.
2. Request a booking.
3. Show customer notification.
4. Log in as organizer.
5. Show organizer booking notification or approval-related notification.
6. Confirm or reject booking.
7. Return as customer and show final result notification.
8. Mark a notification as read.

Notification Targets

The system can create notifications for:

- customer
- organizer
- admin

Examples:

- Customer: "Your booking request is pending."
- Organizer: "A customer requested a booking for your event."
- Admin: "A new booking request was created."
- Customer: "Your booking was confirmed/rejected."

Security Points

- Notification access is protected by JWT.
- Users should only access notifications relevant to them.
- Internal notification creation can be triggered by backend services.
- The service is deployed internally in Azure and accessed through gateway/service communication.
- Secrets and MongoDB URI are environment variables.

DevOps And Cloud Points

- The service has its own Dockerfile.
- The Docker image is published to GHCR.
- The service is deployed to Azure Container Apps.
- The service has a separate SonarCloud project:
  - `EventHub Notification Service`
- The GitHub Actions workflow scans this service using:
  - `sonar/notification-service.properties`

Likely Viva Questions

What is the purpose of the notification service?

It stores and serves in-app notifications for users. It allows customers, organizers, and admins to receive booking-related updates.

How does your service integrate with another microservice?

The booking service calls notification-service to create alerts after booking actions such as request, confirmation, rejection, and cancellation.

Why is notification a separate service?

It separates communication/alert responsibilities from booking logic. This keeps the booking service focused on booking workflow while notification-service owns notification data.

What is mark-as-read used for?

It lets users update notification state after they have seen an alert.

What kind of notifications are created?

Booking request, pending status, organizer approval request, admin alert, confirmation, rejection, and cancellation notifications.

How is your service secured?

It uses JWT-protected routes, environment variables for secrets, and is deployed behind the gateway rather than being exposed directly.

What would you improve in production?

I would add email/SMS integration, real-time notifications with WebSockets, retry handling, and a message queue for reliable async delivery.

Common Group Questions

What is the application?

EventHub is a microservice-based event booking platform. Customers can browse events and request bookings, organizers can manage events and approvals, and admins can oversee activity.

What are the four microservices?

- Auth Service
- Event Service
- Booking Service
- Notification Service

What is the API gateway used for?

The gateway is the public backend entry point. The frontend calls the gateway, and the gateway routes requests to the correct internal service.

What is the main integration flow?

The strongest flow is booking:

1. Customer requests a booking.

2. Booking service calls event service to check and reserve seats.
3. Booking service stores a pending booking.
4. Booking service calls notification service to create alerts.
5. Booking service calls auth service when admin user data is needed.

What DevOps practices are implemented?

- GitHub repository
- GitHub Actions CI
- Docker containerization
- GHCR container image publishing
- Azure Container Apps deployment
- SonarCloud DevSecOps scanning

What security practices are implemented?

- JWT authentication
- Role-based access
- Internal microservice ingress in Azure
- Environment variables and GitHub secrets
- SonarCloud static analysis
- Express Helmet middleware

Quick Emergency Checks

Gateway health:

```
curl https://gateway.mangocoast-efcefaea.southeastasia.azurecontainerapps.io
```

Notification logs:

```
az containerapp logs show --name notification-service --resource-group eventhub-rg
--tail 100
```

Gateway logs:

```
az containerapp logs show --name gateway --resource-group eventhub-rg --tail 100
```